

Cognizant Digital Nurture 5.0

Week 3 – ASP.NET Core Web API

Hands-on Lab Report (Labs 1–6)

Student Name: Ayush Raj

Program: Cognizant Digital Nurture 5.0 – Deep Skilling

Track: ASP.NET Core Web API Development

Module: Week 3 – Hands-on Labs 1 to 6

Technologies Practiced: ASP.NET Core Web API · Swagger (OpenAPI) · JWT Authentication · Entity Relationships · Apache Kafka

Table of Contents

Introduction	3
Lab 1 – Project Setup and Swagger UI Configuration	3
Lab 2 – Testing the Employee API (Unauthenticated Access)	4
Lab 3 – JWT Authentication Implementation	5
Lab 4 – Testing the Employee API (Authenticated Access)	5
Lab 5 – Retrieving a Single Employee by ID	6
Lab 6 – Kafka Integration	6
Kafka Producer Execution	7
Kafka Consumer Execution	7
Conclusion	8

Introduction

This report documents the hands-on labs completed during Week 3 of the Cognizant Digital Nurture 5.0 Deep Skilling program, under the ASP.NET Core Web API track. The objective of this module was to design and build a RESTful Web API using ASP.NET Core, expose and test its endpoints through Swagger (OpenAPI), secure the API using JWT-based authentication, and finally integrate the application with Apache Kafka for asynchronous message-based communication.

The practical work was carried out using Visual Studio Code as the development environment, with the project **WebApi_Handson1** containing Employee and Department entities, a Skills relationship, and a secured Employee API. Apache Kafka (version 2.13-3.9.1) was used on Windows to demonstrate producer–consumer based messaging. Each lab below is supported with screenshots captured directly from the development and testing environment.

Lab 1 – Project Setup and Swagger UI Configuration

Objective: Set up the ASP.NET Core Web API project (WebApi_Handson1) and configure Swagger (OpenAPI) to visualize and document all available API endpoints.

The ASP.NET Core Web API project was executed successfully, and the Swagger UI was launched at **localhost:5206/swagger/index.html**. The Swagger page confirms that the **WebApi_Handson1 v1** API definition (OAS 3.0) is being served correctly, listing the following endpoint groups:

- **Employee** controller – GET, POST /api/Employee and GET, PUT /api/Employee/{id}
- **Values** controller – full CRUD operations (GET, POST, GET/{id}, PUT/{id}, DELETE/{id})
- Default **WebApi_Handson1** group – GET /weatherforecast
- A **Schemas** section defining the Department model and related data contracts

This confirms that the Web API executed successfully and Swagger UI launched correctly, providing a working interface for testing all endpoints.

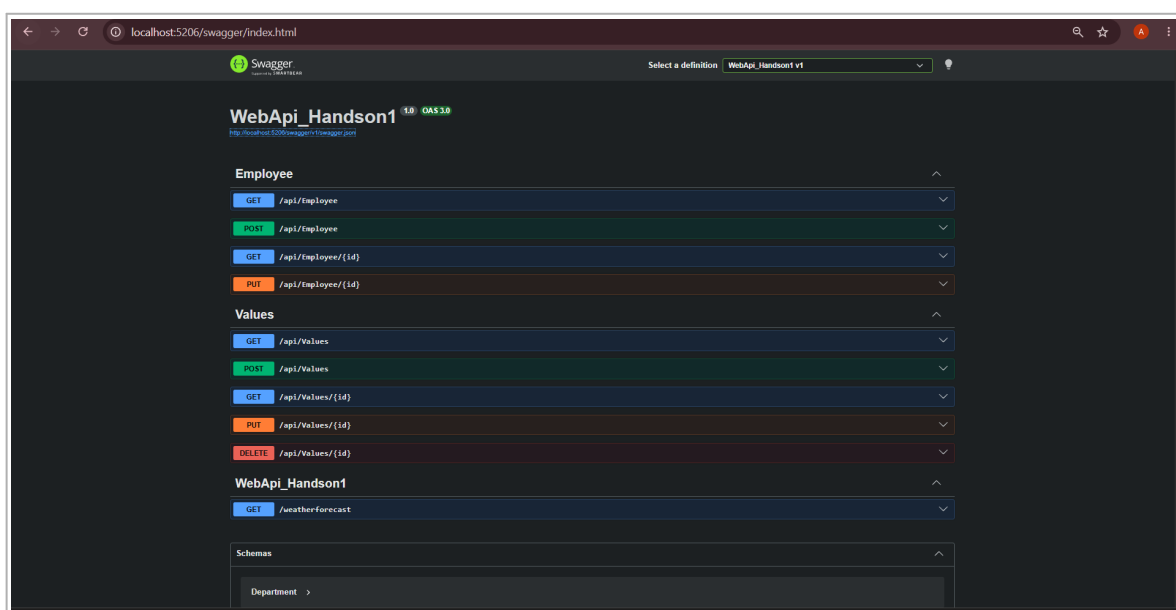


Figure 1.1 – Swagger UI showing the WebApi_Handson1 API with Employee, Values and Schema definitions.

Lab 2 – Testing the Employee API (Unauthenticated Access)

Objective: Test the GET /api/Employee endpoint through Swagger before authentication was enforced, and observe the API's behaviour and error responses.

The GET /api/Employee endpoint was executed directly from Swagger. In an initial test run, the server returned an **HTTP 500 (Internal Server Error)** response. This response was used to verify server-side exception handling behaviour for the Employee endpoint.

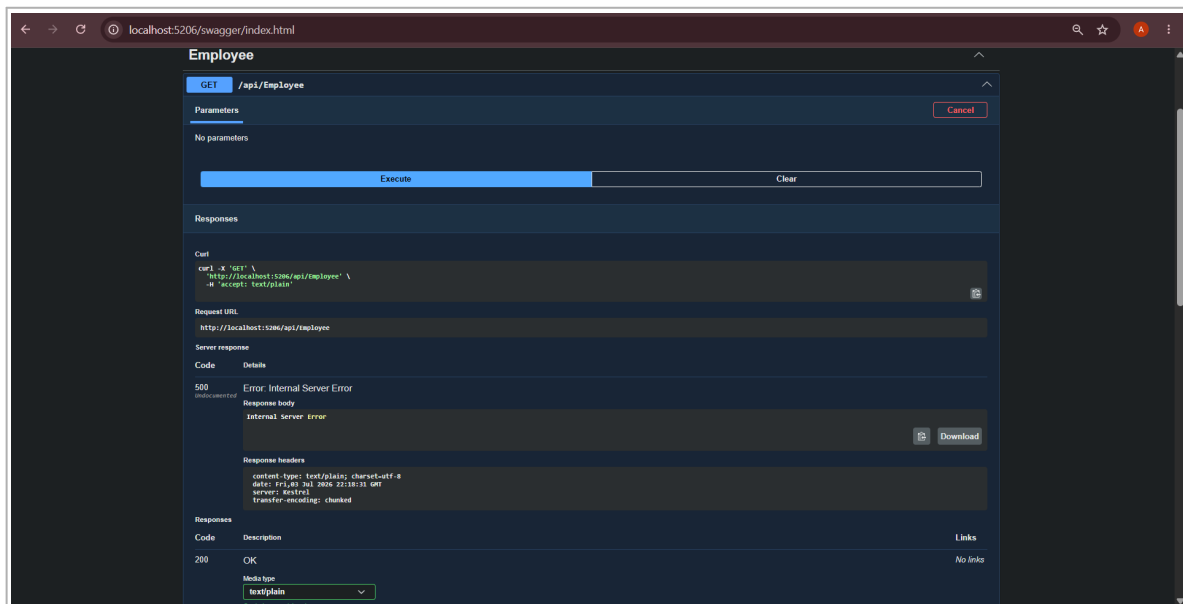


Figure 2.1 – GET /api/Employee returning HTTP 500 (Internal Server Error) response.

In a subsequent test, the same endpoint was called without supplying a JWT token. The API correctly rejected the request with an **HTTP 400 (Bad Request)** response and the message “Invalid request – No Auth token”, confirming that a custom authentication filter was actively protecting the Employee endpoint.

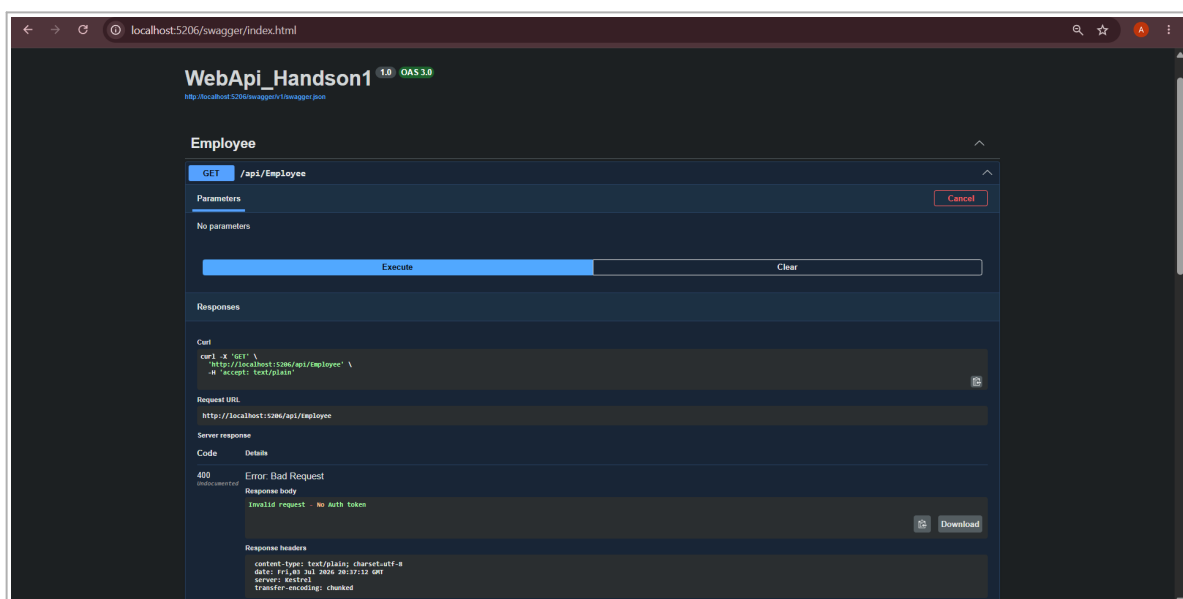


Figure 2.2 – GET /api/Employee returning HTTP 400 Bad Request – “Invalid request - No Auth token”.

Lab 3 – JWT Authentication Implementation

Objective: Implement token-based authentication for the API using an `AuthController` that issues a JSON Web Token (JWT) on login, so that protected endpoints such as `Employee` can be securely accessed.

An `AuthController` was implemented under `WebApi_Handson1.Controllers`. It exposes an `[AllowAnonymous] [HttpGet] Login()` action that generates a JSON Web Token via a `GenerateJSONWebToken(1, "Admin")` method and returns it in the response using `Ok(token)`. The controller uses dependency-injected `IConfiguration` to read the token settings required for signing the JWT. This authentication mechanism supplies the token that must be sent with subsequent requests to protected endpoints such as `Employee`.

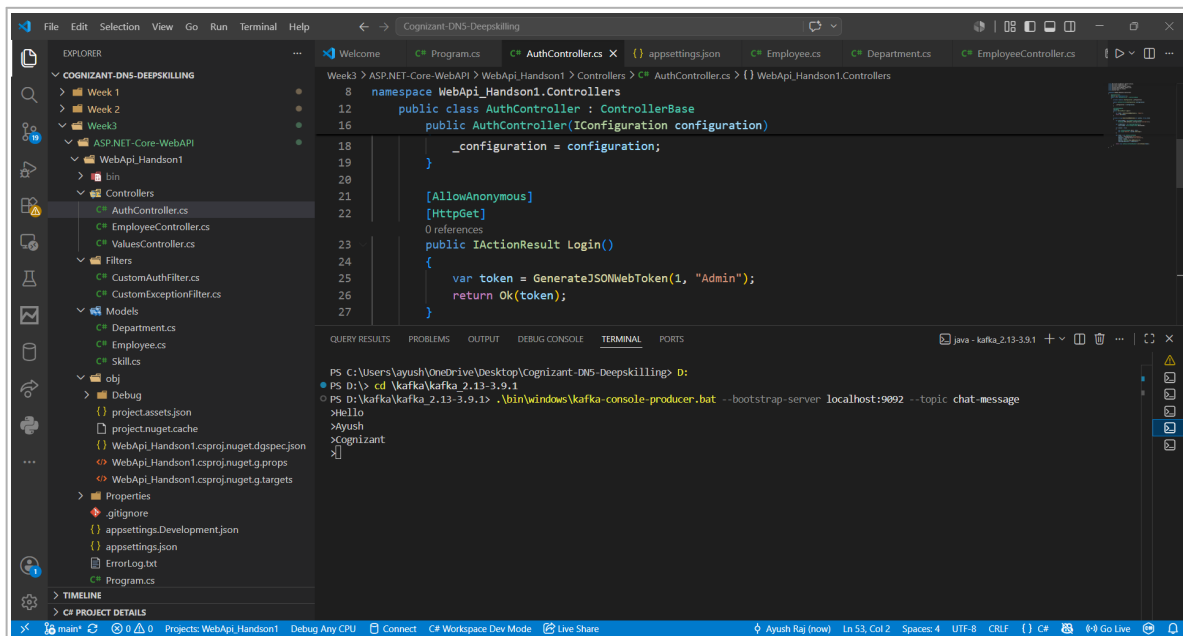


Figure 3.1 – `AuthController.cs` implementing the `Login()` action that generates a JWT using `GenerateJSONWebToken()`.

Lab 4 – Testing the Employee API (Authenticated Access)

Objective: Re-test the GET `/api/Employee` endpoint after supplying a valid JWT, and verify that the API returns the expected employee data.

With a valid JWT token generated from the `AuthController` supplied to the request, the GET `/api/Employee` endpoint was executed again from Swagger. The API returned a successful **HTTP 200 OK** response containing a JSON array of employee records. Each record includes the employee's id, name, salary, permanent status, an embedded department object (id and name), and a list of associated skills — confirming that the `Employee`, `Department` and `Skill` entities are correctly related and serialized.

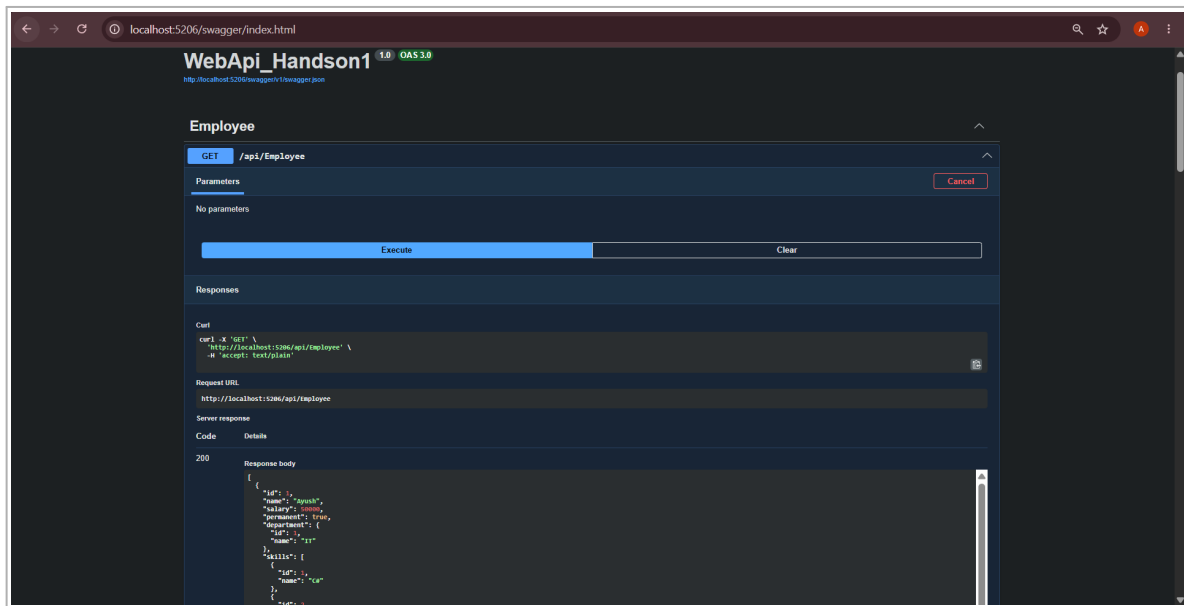


Figure 4.1 – GET /api/Employee returning HTTP 200 OK with the full list of employees after authentication.

Lab 5 – Retrieving a Single Employee by ID

Objective: Test the GET /api/Employee/{id} endpoint to retrieve a single employee record and validate the detailed JSON structure returned by the API.

The endpoint GET /api/Employee/1 was executed successfully, returning an **HTTP 200 OK** response. The response body confirms an employee record for “**Ayush**” with a salary of 50000, permanent status of true, department “**IT**”, associated skills “**C#**” and “**.NET**”, and a dateOfBirth value of 2003-01-01T00:00:00. This confirms that the API correctly retrieves and serializes a single employee along with its related department and skills data.

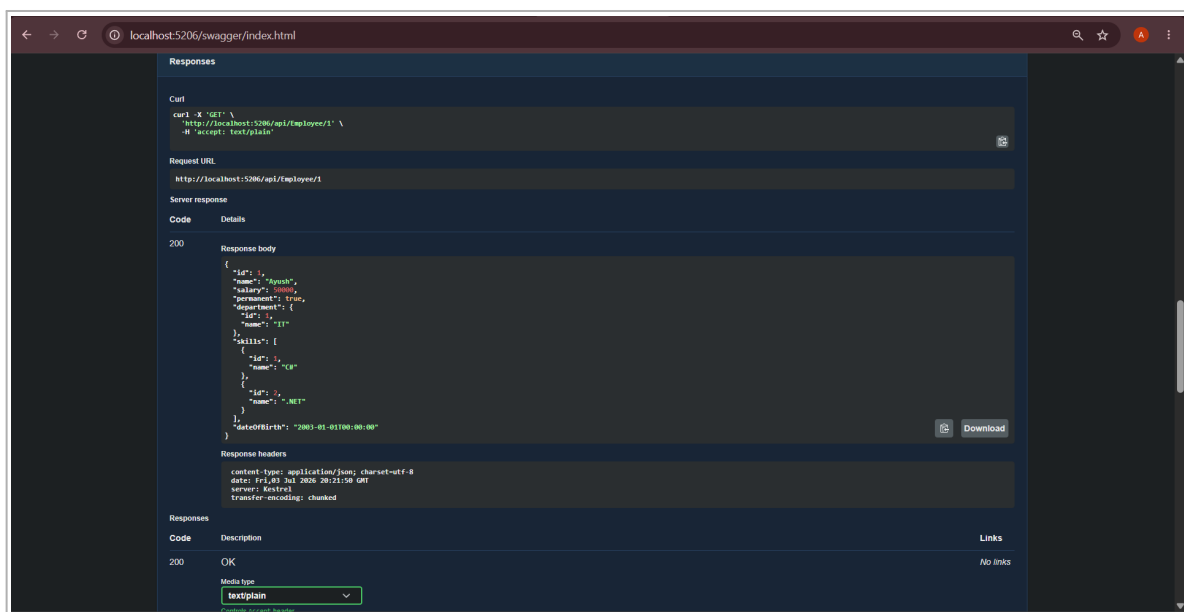


Figure 5.1 – GET /api/Employee/1 returning HTTP 200 OK with detailed employee, department and skills data.

Lab 6 – Kafka Integration

Objective: Set up Apache Kafka (v2.13-3.9.1) on Windows and demonstrate asynchronous message-based communication using a Kafka producer and a Kafka consumer on the **chat-message** topic.

Prior to running the producer and consumer, the ZooKeeper server and the Kafka broker were started successfully from the Kafka installation directory, providing the coordination and messaging backbone required for topic-based communication. The **chat-message** topic was then used for producing and consuming messages, as shown below.

Kafka Producer Execution

The Kafka console producer was started using **kafka-console-producer.bat --bootstrap-server localhost:9092 --topic chat-message**. Three messages – “Hello”, “Ayush” and “Cognizant” – were published to the topic from the terminal, confirming that the producer was able to connect to the broker and publish messages successfully.

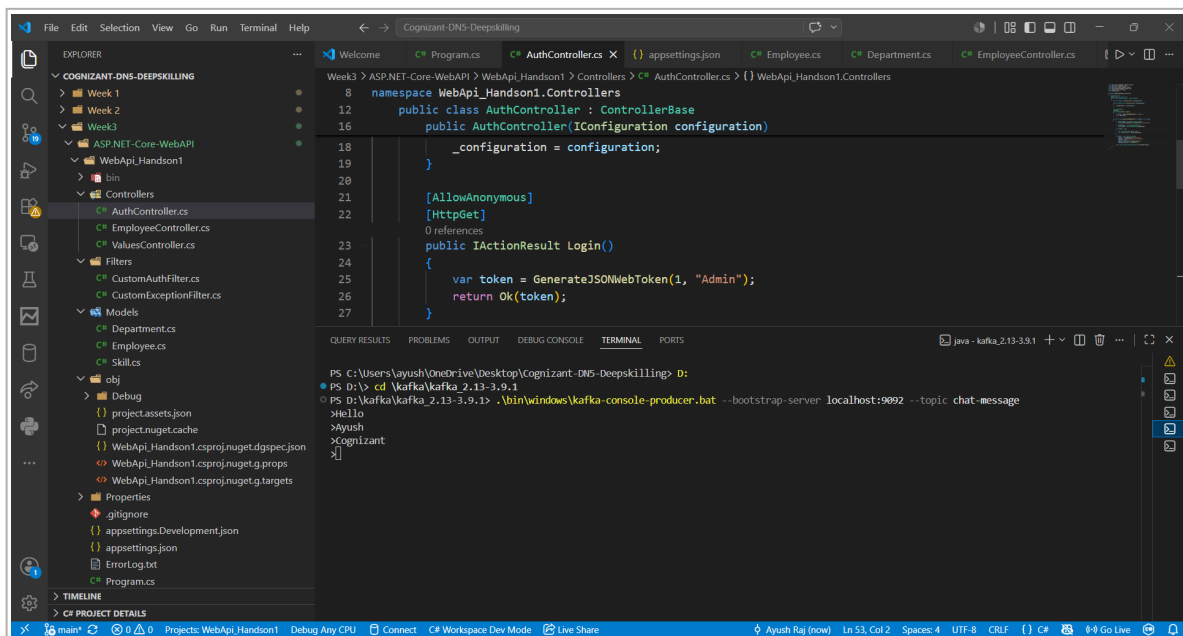


Figure 6.1 – Kafka Producer terminal publishing “Hello”, “Ayush” and “Cognizant” to the chat-message topic.

Kafka Consumer Execution

The Kafka console consumer was started using **kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic chat-message --from-beginning**. The consumer successfully received and displayed all three messages (“Hello”, “Ayush”, “Cognizant”) in the same order they were produced, confirming successful end-to-end message transmission between the Kafka producer and consumer.

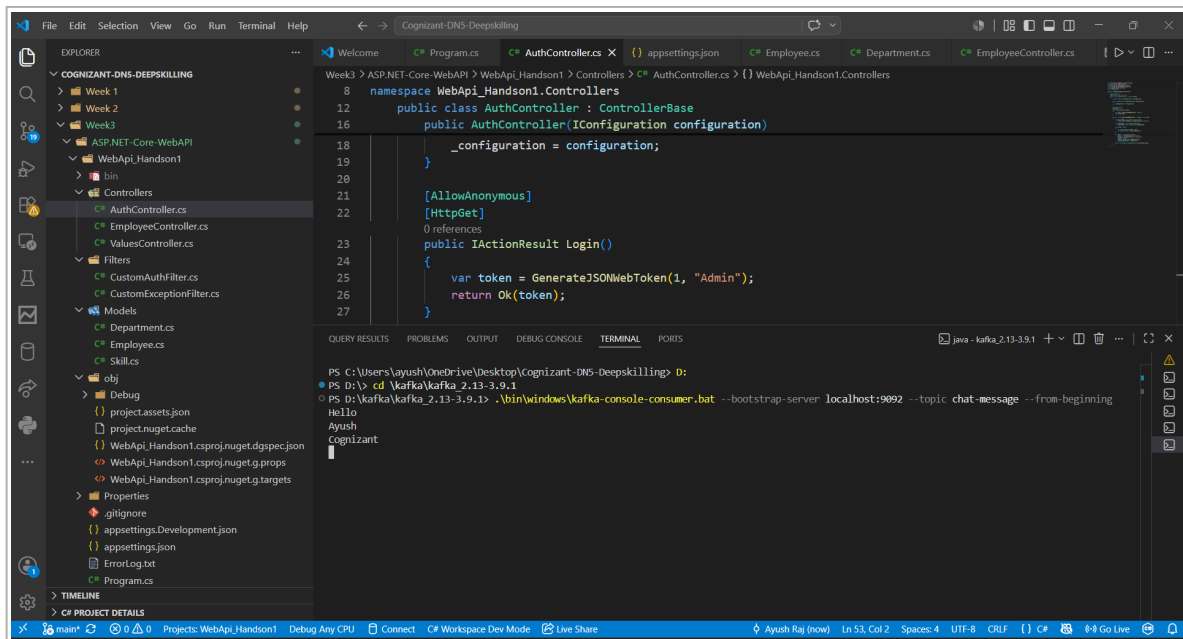


Figure 6.2 – Kafka Consumer receiving “Hello”, “Ayush” and “Cognizant” from the chat-message topic.

This confirms that ZooKeeper and the Kafka broker were started successfully, the chat-message topic was created and used correctly, and the producer–consumer message flow completed successfully with no data loss.

Conclusion

Labs 1 through 6 of the Week 3 ASP.NET Core Web API module were completed successfully. The WebApi_Handson1 project was built, executed, and exposed through Swagger UI, allowing all Employee, Values and Department endpoints to be inspected and tested interactively. The Employee API was progressively secured using JWT-based authentication implemented in AuthController, and its behaviour was validated across unauthenticated (400/500) and authenticated (200 OK) scenarios, including retrieval of both the full employee list and an individual employee record with nested department and skills data.

In the final lab, Apache Kafka was configured and used to demonstrate real-time, asynchronous communication through a producer–consumer model on the chat-message topic, with messages successfully transmitted from the producer terminal and received in the correct order by the consumer terminal.

Overall, this module provided practical, hands-on experience with ASP.NET Core Web API development, REST API design and testing, Swagger/OpenAPI documentation, JWT authentication and authorization, entity relationships (Employee–Department–Skill), and Apache Kafka-based message streaming – core skills applicable to building secure, scalable backend and microservice-based systems.